

1. ALGORITMI & SKLEARN CHEATCODES

Naive Bayes BoW (counts):

```
MultinomialNB().fit(X, y)
→  $\log P(c|x) = \log P(c) + \sum \log P(x_i|c)$ 
```

KNN:

```
KNeighborsClassifier(n=k, metric='l2')
→ vot majoritar din k vecini, k impar
```

BoW char-level:

```
CountVectorizer(analyzer='char')
→  $X = \text{bow.fit\_transform}(\text{train}).\text{toarray}()$ 
```

StandardScaler:

```
StandardScaler() – fit pe train
→  $z = (x - \mu) / \sigma$  pe coloane
```

L1 / L2 norm:

```
Normalizer(norm='l1' | 'l2')
→  $x / \sum |x_i|$  sau  $x / \sqrt{\sum x_i^2}$  pe linii
```

SVM linear:

```
SVC(kernel='linear', C=...)
→ max margin, hinge loss +  $\frac{1}{2}\|w\|^2$ 
```

SVM RBF:

```
SVC(kernel='rbf', C=..., gamma='scale')
→  $K = \exp(-\gamma\|x-y\|^2)$  — default robust
```

Ridge:

```
Ridge(alpha=...)
→  $\text{MSE} + \alpha\|w\|^2$  — L2 regularization
```

Lasso:

```
Lasso(alpha=...)
→  $\text{MSE} + \alpha\|w\|_1$  — sparse ponderi
```

Kernel Ridge:

```
KernelRidge(alpha=..., kernel=...)
→ Ridge în spațiu dual; kernel='precomputed' OK
```

MLP:

```
MLPClassifier(hidden_layer_sizes=(100,))
→ activation='relu', solver='adam'
```

4. DECISION TREE: CE MODEL ALEGI

PROBLEMA E REGRESIE

features colineari/multe → Ridge(alpha=1)
features puține / clean → LinearRegression
vrei să reții doar ce contează → Lasso

PROBLEMA E CLASIFICARE

text/document → BoW char + NB sau Ridge OvA
histograme / contoare → Hellinger sau Intersection
vectori dense, low-dim → SVM RBF
imagini 28×28 (MNIST) → KNN L2 sau MLP
date deja separabile liniar → SVM linear / Ridge OvA

STUCK?

default: SVM RBF cu C=10, gamma='scale'
pentru raport: sweep [0.1, 1, 10, 100] pe C/α

5. HYPERPARAMETER SURVIVAL GUIDE

α (Ridge/KRR) 0.01 → 1000 (semilog)
→ mai mare = ponderi mai mici

C (SVM) 0.01 → 1000 (semilog)
→ mare = strict, mic = relaxat

γ (RBF) 'scale' default
→ sau 0.001 → 1 (semilog)

k (KNN) 1, 3, 5, 7, 9 (impar)
→ mic = overfit, mare = smooth

num_bins (NB) 3, 5, 7, 9, 11
→ 5-7 e sweet-spot tipic

hidden (MLP) (50,), (100,), (50,30)
→ mai mare = mai mult overfit

η (perceptron) 0.001 → 0.5
→ mare → diverge

threshold conv 0.9 sau 0.99
→ 0.99 = match-uri quasi-exacte

6. SANITY CHECKS — RULAȚI MEREU

- ▶ print(X_train.shape, X_test.shape) după FIECARE transform
- ▶ verifică distribuția claselor: np.bincount(y_train)
- ▶ testează modelul cel mai simplu mai întâi (NB / Ridge)
- ▶ salvează predicțiile cu np.save() imediat ce le ai
- ▶ verifică acuratețea pe train (overfitting?) vs test
- ▶ dacă acuratețea = 1.0 pe test, ai un bug

2. KERNEL — FORMULE RAPID

Linear	$K(x, y) = x^T y$
Polynomial d	$(x^T y + c)^d$
RBF / Gaussian	$\exp(-\gamma \ x - y\ ^2)$
Sigmoid	$\tanh(\alpha x^T y + c)$
Hellinger	$\sum \sqrt{x_i} \cdot \sqrt{y_i} \quad (x, y \geq 0)$
Intersection	$\sum \min(x_i, y_i) \quad (x, y \geq 0)$

3. ONE-VS-ALL PATTERN

```
def ova(Xtr, ytr, Xte, model_cls, **kw):
    n = int(ytr.max()) + 1
    preds = []
    for c in range(n):
        bin_y = ((ytr == c) * 2) - 1
        m = model_cls(**kw).fit(Xtr, bin_y)
        preds.append(m.predict(Xte))
    return np.argmax(np.array(preds), 0)
```

FORMAT COLOCVIU (V1 + V2)

Ex	Algoritm V1	Algoritm V2	Barem	Target acc	Hiperparam
Ex1	Naive Bayes BoW char	Ridge BoW char	3.5p	$\geq 70\%$	$\alpha=1$ (Ridge)
Ex2	Convoluție 1D t=0.9	Convoluție 1D t=0.99	1.5p	implicit	threshold
Ex3	KRR linear	SVM RBF	2.5p	$\geq 65\%$	$\alpha=100$ / $C=100$
Ex4	SVM Hellinger (prec)	KRR Intersection (prec)	2p	$\geq 60\%$	$C=10$ / $\alpha=100$
Ex5	Raport hyperparam sweep	Raport hyperparam sweep	1.5p	—	PDF max 2pag
Of.	Oficiu	Oficiu	1p	—	—

CONVOLUȚIE 1D CU 3-GRAME (EX2) — OBLIGATORIE

```
def convolve(sample, kernels):
    # sample (L,), kernels (K, n)
    K, n = kernels.shape; L = sample.shape[0]
    if L < n: return np.zeros((K, 1))
    out = np.zeros((K, L - n + 1))
    kn = np.linalg.norm(kernels, axis=1)
    for i in range(L - n + 1):
        win = sample[i:i+n]; wn = np.linalg.norm(win)
        if wn == 0: continue
        out[:, i] = (win * kernels).sum(1) / (kn * wn + 1e-12)
    return out # apoi: (out > t).sum(axis=1)
```

KERNELI PRECOMPUTED — PATTERN EX3/EX4

```
def hellinger(X, Y):
    return np.sqrt(np.maximum(X, 0)) @ np.sqrt(np.maximum(Y, 0)).T

def intersection(X, Y):
    out = np.zeros((len(X), len(Y)))
    for i, x in enumerate(X):
        out[i] = np.minimum(x[None, :], Y).sum(axis=1)
    return out

K_train = kernel_fn(X_tr, X_tr); K_test = kernel_fn(X_te, X_tr)
SVC(kernel='precomputed', C=10).fit(K_train, y_train).predict(K_test)
```

CAPCANELE CLASICE (CITEȘTE ÎNAINTE DE EXAMEN!)

- Folosești documents în loc de data la convoluție.**
✓ Convoluția cere VECTORI numerici. Folosește train_reader.data.
- Faci fit_transform pe TEST.**
✓ Fit pe TRAIN. Transform pe ambele. Niciodată invers.
- Setezi parametrii alegându-i pe baza acc pe TEST.**
✓ Folosește train_test_split pe TRAIN + cross_val_score. Test = blind.
- Uți kernel='precomputed' cu kernel custom.**
✓ Sklearn crede că matricea ta e feature matrix — rezultat aiurea.
- Uți să faci one-vs-all pentru Ridge / KernelRidge la clasificare.**
✓ Ridge/KRR sunt REGRESORI (un singur scor continuu). SVC face OvO singur. Pentru Ridge/KRR scrii tu OvA + argmax.
- Salvezi predicții cu nume greșit.**
✓ Format obligatoriu: Nume_Prenume_Grupa_subiect1_solutia.npy
- Trimiți raportul fără plot-uri sweep.**
✓ Cel puțin 1 grafic accuracy vs hyperparam. Etichete pe axe. Caption.